47

BÖLÜM: KUBERNETES OPERATORS – KENDI AKILLI ROBOTUNU KODLA | MODULE: KUBERNETES OPERATORS – BUILD YOUR OWN ROBOT SRE

Java ile Operator Yazımı: Java Operator SDK (JOSDK) Temelleri

Building Operators in Java: Java Operator SDK (JOSDK)

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Dünyadaki herkes size 'Kubernetes operatörleri sadece Go (Golang) ile yazılır' diye yalan söyleyebilir. İnanmayın! Bir Java ve Spring Boot geliştiricisi olarak, kendi en iyi bildiğiniz dilde K8s operatörü yazabilirsiniz. Java Operator SDK (JOSDK) sayesinde tek yapmanız gereken bir Java arayüzünü (`Reconciler<SpringBootApp>`) uygulamak ve içine kodunuzu yazmaktır!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

The tech world might tell you that Kubernetes operators can ONLY be coded in Go. Don't believe the myth! As a Java and Spring Boot engineer, you can build production-grade operators in your favorite language. With Java Operator SDK (JOSDK), you simply implement a clean Java interface (`Reconciler<SpringBootApp>`) and write standard object-oriented code!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Java Operator SDK (JOSDK), Fabric8 Kubernetes Client üzerine kurulmuş modern bir framework'tür. Quarkus veya Spring Boot ile entegre çalışır. `@ControllerConfiguration` anotasyonuyla CRD'ye bağlanır. Reconciler arayüzü tek bir metod içerir: `UpdateControl<SpringBootApp> reconcile(SpringBootApp resource, Context<SpringBootApp> context)`. JOSDK arka planda thread havuzunu, Watch eventlerini, retry mekanizmalarını ve exponential backoff algoritmalarını otomatik yönetir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Java Operator SDK (JOSDK) is a high-performance framework built atop Fabric8 Kubernetes Client, integrating seamlessly with Spring Boot and Quarkus. By implementing `Reconciler<T>` and annotating with `@ControllerConfiguration`, JOSDK manages informers, work queues, event filtering, and exponential backoff retry semantics transparently.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```java
// Java Operator Reconciler İskeleti:
@ControllerConfiguration(name = "springbootapp-controller")
public class SpringBootAppReconciler implements Reconciler<SpringBootApp> {

    private final KubernetesClient client;

    public SpringBootAppReconciler(KubernetesClient client) {
        this.client = client;
    }

    @Override
    public UpdateControl<SpringBootApp> reconcile(SpringBootApp resource,
Context<SpringBootApp> context) {
        String appName = resource.getMetadata().getName();
        String targetImage = resource.getSpec().getImage();
        int replicas = resource.getSpec().getReplicas();

        // Akıllı Robot Mantığı: Deployment var mı kontrol et, yoksa yarat!
        reconcileDeployment(resource, appName, targetImage, replicas);

        return UpdateControl.noUpdate();
    }
}
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Java ile operatör yazarken Docker imajınızın bellek tüketimine dikkat edin. Standart bir HotSpot JVM operatör için 200MB RAM harcayabilir. GraalVM Native Image derlemesi yaparak operatörünüzü 20MB RAM ile 0.01 saniyede açılan bir ikili dosyaya dönüştürebilirsiniz!

🇬🇧 WATCH OUT

Standard JVM runtime footprints hover around 200MB. For optimal lightweight operator footprints, compile your Java operator with GraalVM Native Image.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- Java geliştiricileri Go öğrenmek zorunda kalmadan yerel K8s operatörleri yazabilir.
- Fabric8 client K8s API'sindeki tüm nesneleri tip güvenli (type-safe) Java sınıflarına çevirir.
- Quarkus ve Spring Boot entegrasyonu modern geliştirme konforu sunar.

- Java engineers can build high-performance K8s operators without mastering Go.
- Fabric8 provides type-safe Java POJOs for all native and custom K8s resources.
- Seamless integration with Spring Boot enables dependency injection and metrics.